DAG Authoring Guide

A practical, end-to-end guide to hand-writing a **workflow DAG** as JSON for the BigBrain workflow engine — defining the tasks, wiring the graph, and passing data between steps without going through the LLM planner.

This is the "how do I write the JSON myself" companion to the reference material:

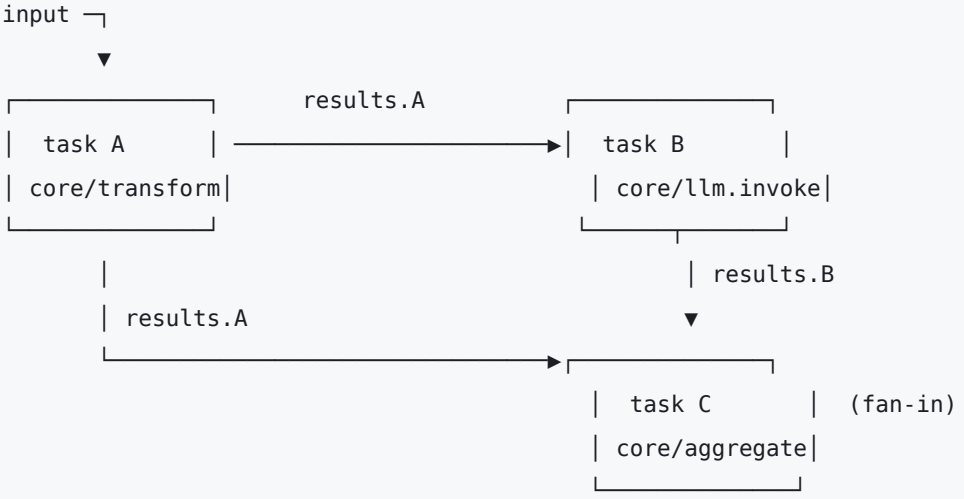
Doc	What it covers
This guide	Authoring a workflow definition by hand: shape → tasks → dependencies → data bindings → validation
`CAPABILITY_CATALOG.md`	The catalog of capability types and their input/output schemas
`NEURON_ARCHITECTURE.md`	How tasks are routed to neurons and executed
`WORKFLOW_EVENTS.md`	The lifecycle events a running workflow emits

If you want a workflow generated for you from a natural-language goal, use the planner (`POST /plan`). Read this guide when you want to author the DAG **directly** — for deterministic pipelines, templates, fixtures, or tests.

---

1. The mental model

A workflow is a **directed acyclic graph (DAG) of tasks**. Each task names a **capability** to run, declares which upstream tasks it **depends on**, and maps data from the shared workflow **context** (and from upstream outputs) into its own input. The engine schedules a task once its dependencies are satisfied, routes it to a handler/neuron by capability type, validates its input and output against the capability's schemas, and writes the result back into the context where downstream tasks can read it.



You author three things: the **tasks**, the **edges** between them (`dependencies`), and the **data flow** (`contextMapping`). The engine owns everything else.

---

2. The top-level shape

A workflow definition is a JSON object with exactly three required keys:

```
{
  "tasks": {  },
  "context": { "input": {  }, "results": {}, "shared": {} },
  "artifacts": {}
}
```

Key	Type	Meaning
`tasks`	`Record<TaskId, Task>`	The DAG. Keyed by task id; **the key must equal the task's own `id`**. At least one task.
`context`	object	The shared data space (below). **Required** — the engine writes task outputs here, so the object must exist.
`artifacts`	`Record<ArtifactId, ArtifactReference>`	Large/by-reference outputs. **Required**, usually `{}` at authoring time.

`context` has three sub-spaces, all required:

```
"context": {
  "input":   { "language": "python", "code": "..." },
  "results": {},
  "shared":  {}
}
```

- **`input`** — workflow-level input you provide at creation. This is *your* parameter object; bind into tasks via `input.<field>`.
- **`results`** — accumulated task outputs, keyed by task id. **Start it empty (`{}`)**; the engine appends `results.<taskId>` as each task completes.
- **`shared`** — scratch space for values that aren't a single task's output. Start empty unless you're seeding shared state.

> The `tasks` map key and the task's `id` field must match. `"prepare_context": { "id": "prepare_context", … }`. A mismatch is a definition error.

---

3. Anatomy of a task

Every task is an object. Here is the full field set; most are optional, and several are runtime fields the engine manages after creation.

```
"prepare_context": {
  "id": "prepare_context",
  "description": "Build the review prompt from user input",
  "requiredCapability": "core/transform",

  "dependencies": [],
  "dependencyCondition": { "type": "all_succeeded" },

  "contextMapping": {
    "include": ["input"],
    "inputMappings": {}
  },
  "input": {
    "expressionType": "jsonata",
    "expression": "{ 'prompt': 'Review this code', 'code': input.code }"
  },

  "status": "pending",
  "attempts": 0,
  "createdAt": "2026-01-01T00:00:00Z",
  "updatedAt": "2026-01-01T00:00:00Z"
}
```

Fields you author

Field	Required	Meaning
`id`	✅	Task identifier. Any **non-empty string** (the persist schema only requires `minLength: 1`). Must equal the `tasks` map key. See the id-convention note below.
`requiredCapability`	▲ technically optional, **always set it**	The capability type that routes this task to a handler, e.g. `core/transform`, `core/llm.invoke`, `system/bash.exec`. Without it the task can't be dispatched.
`dependencies`	✅	Array of upstream task ids. `[]` = a root task.
`dependencyCondition`	optional	When upstreams count as satisfied. Defaults to `all_succeeded`. See §4.
`contextMapping`	✅	How data flows into this task. `{ include: [...], inputMappings?: {...} }`. See §5.
`input`	optional	Static, literal input for the capability. Merged with the bindings from `contextMapping` (bindings win on key collision).
`description`	optional	Human-readable purpose. Recommended.
`onBehalfOf`	optional	User id for on-behalf-of (user-scoped) capabilities.
`inputSchema` / `outputSchema`	optional	JSON Schema snapshotted from the catalog at plan time. You normally omit these by hand; the engine attaches them from the live catalog at dispatch.
`retry`	optional	Opt-in retry policy (below). Absent = fail-fast.
`loop`	optional	Marks the task as a **loop controller**: the engine runs it repeatedly (pagination or per-item fan-out) and completes it with an aggregate. See §6.

Initial-state fields

The task schema also carries runtime state. At authoring time set them to their initial values:

```
"status": "pending",
"attempts": 0,
"createdAt": "<ISO-8601>",
"updatedAt": "<ISO-8601>"
```

`status` starts at `"pending"`; `attempts` at `0`. `output`, `error`, `leaseId`, `leaseExpiresAt`, `startedAt`, `completedAt` are populated by the engine — leave them out (or `null`). The full status set the engine moves a task through is: `pending → ready → assigned → running → completed` (or `failed` / `skipped` / `cancelled`).

Retry policy (optional)

```
"retry": {
  "maxRetries": 2,
  "backoffMs": 1000,
  "backoffMultiplier": 2,
  "maxBackoffMs": 10000
}
```

Retries are **opt-in per task**. Omit `retry` for fail-fast behaviour. Note the engine only retries failures the handler classifies as retryable.

> **Task id convention.** The core persist schema accepts any non-empty string, which is why real workflows use hyphenated ids (`prepare-context`) and even ULIDs. **However**, the agentic planner enforces a stricter slug — `^[a-z][a-z0-9_]{0,63}$` (lowercase, digits, underscores; starts with a letter; ≤ 64 chars). If your DAG might be extended or patched by the agentic path, use that stricter form (underscores, no hyphens) to stay compatible.

---

4. Wiring the graph

Edges are **explicit** via each task's `dependencies` array — a list of upstream task ids. A task with `"dependencies": []` is a root and runs immediately; a task that lists two upstreams is a fan-in.

```
"aggregate": {
  "id": "aggregate",
  "dependencies": ["llm_anthropic", "llm_openai"],
  "dependencyCondition": { "type": "all_completed" }
}
```

`dependencyCondition.type` controls *when* the dependencies count as satisfied:

Condition	Fires when…	Use for
`all_succeeded` *(default)*	every dependency completed **successfully**	the normal case
`all_completed`	every dependency reached a terminal state (success **or** failure)	fan-in/aggregation that tolerates partial failure
`any_succeeded`	at least one dependency succeeded	race / first-good-wins
`any_completed`	at least one dependency finished	barriers
`any_failed`	at least one dependency failed	error-handler / compensation branches

Acyclicity is enforced. The graph must be a DAG; a cycle is rejected at validation (DFS cycle check). Also: any id you reference in `dependencies` or in a binding must be a task that actually exists in the `tasks` map.

> Data bindings imply edges too. If task B binds `results.A.*`, the engine treats B as depending on A even beyond the explicit `dependencies` list — but **author the explicit edge anyway**: it documents intent and keeps the graph readable.

---

5. Passing data between tasks

This is the heart of authoring. A task assembles its input from two places, both inside `contextMapping`:

```
"contextMapping": {
  "include": ["results.prepare_context", "input"],
  "inputMappings": {
    "prompt": "results.prepare_context.prompt",
    "language": "input.language"
  }
}
```

- **`include`** — the context sub-trees this task is allowed to read. Can be a root (`"input"` , `"results"` , `"shared"`) or a narrower path (`"results.prepare_context"`). Think of it as the read scope.
- **`inputMappings`** — the actual wiring: a map of **consumer key → context dot-path**. Each entry pulls a value out of the context and places it on the task's input under the consumer key.

Dot-path roots

A binding's right-hand side is a dot-path rooted at one of exactly three roots:

Root	Resolves to	Example
`results.<taskId>.<field>…`	an upstream task's output	`results.llm_anthropic.content`
`input.<field>…`	workflow-level input	`input.code`
`shared.<field>…`	shared scratch state	`shared.run_id`

Any other root is rejected (`INVALID_CONTEXT_PATH_ROOT`).

Nested binding keys

The **consumer key** (left-hand side) may itself be a dot-path, which assembles a nested object on the input:

```
"inputMappings": {
  "value.user": "results.run_whoami.stdout",
  "value.when": "results.run_date.stdout"
}
```

produces the input fragment:

```
{ "value": { "user": "<stdout>", "when": "<stdout>" } }
```

You can also index array positions this way (`"responses.0"` , `"responses.1"`) to build an array from multiple upstream outputs.

Static input + merge rule

`input` holds literal values the capability needs that aren't bound from context:

```
"input": { "model": "claude-sonnet-4-20250514", "maxTokens": 2048 },
"contextMapping": {
  "include": ["results.prepare_context"],
  "inputMappings": { "prompt": "results.prepare_context.prompt" }
}
```

The engine merges the two. **On a key collision, the `contextMapping` binding wins** over the static `input` value. Effective input for the task above: `{ model, maxTokens, prompt }` .

---

6. Loops (bounded repetition)

When one step must run an unknown-but-bounded number of times — paginate an API to exhaustion, or run once per element of a list another task produced — attach a `loop` spec to a **single task** instead of hand-building a chain or fan-out. A task carrying `loop` is a **controller**: it never executes itself. The engine materializes each round as a *real* task in the DAG (`<id>__i1` , `<id>__i2` , …), each with its input frozen at materialization (the controller's base `input` ⊕ the carried cursor / page number / item). Because iterations are ordinary tasks, they get the ordinary dispatch, policy, retry, and audit treatment; when the loop terminates, the controller completes with an **aggregate** of the iterations' outputs, published to `results.<id>` like any other task result.

The three kinds

Kind	Repeats until…	Carries between rounds
`cursor`	a next-page token stops appearing in the output	the token → `cursorParam`
`pages`	a page comes back empty	the page number → `pageParam`
`map`	every item of an upstream array is processed	one array element → `itemParam`

`cursor` — token pagination.

```
"all_issues": {
  "id": "all_issues",
  "requiredCapability": "mcp/linear/list_issues",
  "dependencies": ["resolve_team"],
  "contextMapping": { "include": [] },
  "input": { "team": "Holokai", "limit": 250 },
  "loop": {
    "kind": "cursor",
    "cursorRef": "iteration.content.cursor",
    "cursorParam": "cursor",
    "maxIterations": 25,
    "collectRef": "content.issues"
  },
  "status": "pending", "attempts": 0,
  "createdAt": "2026-01-01T00:00:00Z", "updatedAt": "2026-01-01T00:00:00Z"
}
```

(The `resolve_team` dependency isn't decoration — a controller must not be a root task; see the caveats below.) Iteration 1 runs with the base input. After each iteration the engine resolves `cursorRef` **inside that iteration's output** (the `iteration.` root is mandatory for `cursorRef` / `emptyRef`) and, if it found a token, materializes the next iteration with the token placed at `cursorParam`. Terminates `condition` when the ref resolves to `undefined` / `null` / `''` / `false` (`0` continues), `no_progress` if the same token repeats twice (fixpoint guard), or `bound` at `maxIterations`.

`pages` — numbered pagination. The engine sets `pageParam` to `start`, `start + 1`, … (default `start` = 1) and terminates `condition` on the first page where `emptyRef` resolves to an empty array, `undefined`, `null`, or `false`. Note `0` and `''` do **not** count as empty (unlike `cursor`'s token test) — point `emptyRef` at the page's *items array*, not at a count field, or the loop runs to `maxIterations`.

```
"loop": { "kind": "pages", "pageParam": "page", "start": 1,
          "emptyRef": "iteration.content.items", "maxIterations": 10 }
```

`map` — per-item fan-out. `itemsRef` names a workflow-context dot-path (rooted `results.` / `input.` / `shared.`) resolving to an array; the engine runs one iteration per element, placing the element — or its `itemPath` projection — at `itemParam`:

```
"issue_details": {
  "id": "issue_details",
  "requiredCapability": "mcp/linear/get_issue",
  "dependencies": ["search"],
  "contextMapping": { "include": ["results.search"] },
  "input": { "includeRelations": true },
  "loop": { "kind": "map", "itemsRef": "results.search.content.issues",
            "itemPath": "id", "itemParam": "id",
            "maxItems": 50, "maxConcurrency": 4 },
  "status": "pending", "attempts": 0,
  "createdAt": "2026-01-01T00:00:00Z", "updatedAt": "2026-01-01T00:00:00Z"
}
```

`cursor` and `pages` are strictly sequential — the next iteration task is only created after the previous one finishes (from iteration 2 on, each iteration also lists its predecessor in its `dependencies`). `map` runs a sliding window of up to `maxConcurrency` iterations at once (default 1).

Spec fields

Field	Kinds	Required	Meaning
`kind`	—	✅	`'cursor' | 'pages' | 'map'`
`cursorRef`	cursor	✅	Dot-path to the next token, rooted `iteration.`
`cursorParam`	cursor	✅	Input field that receives the carried token
`pageParam`	pages	✅	Input field that receives the page number
`start`	pages	–	First page number (default 1)
`emptyRef`	pages	✅	Dot-path (rooted `iteration.`) that is an empty array / `null` / `undefined` / `false` on the last page (`0` / `''` don't stop the loop)
`itemsRef`	map	✅	Context dot-path (rooted `results.` / `input.` / `shared.`) to the item array
`itemPath`	map	–	Dot-path *into* each item (omit to pass the whole item)
`itemParam`	map	✅	Input field that receives each item
`maxConcurrency`	map	–	Parallel-iteration ceiling (default 1)
`maxIterations`	cursor, pages	✅	Hard iteration ceiling. **Rejected on `map`** — items are bounded by `maxItems`
`maxItems`	map	✅	Hard ceiling on items processed
`collectRef`	all	–	Dot-path *into* each iteration output to keep in the aggregate (omit to keep the full output). Use it to keep aggregates small
`maxCollectedBytes`	all	–	Aggregate size cap in bytes (authored range 1 KiB – 10 MiB); exceeding it stops the loop as `bound` with partials preserved. **Omitting it does not mean uncapped** — the deployment default (256 KiB) applies

On the agentic patch path bounds are **mandatory and platform-capped** by the validator. At run time the engine independently defaults `maxIterations` and `maxCollectedBytes` when absent — but nothing clamps `maxItems` or `maxConcurrency` outside the validator (see the caveats below). The validator ceilings are env-overridable per deployment:

Cap	Default	Env override
`maxIterations` ≤	100	`BIGBRAIN_LOOP_MAX_ITERATIONS`
`maxItems` ≤	500	`BIGBRAIN_LOOP_MAX_ITEMS`
`maxConcurrency` ≤	4	`BIGBRAIN_LOOP_MAX_CONCURRENCY`
`maxCollectedBytes` default	262144 (256 KiB)	`BIGBRAIN_LOOP_MAX_COLLECTED_BYTES`

The aggregate

On termination the controller's output — and, on success, its `results.<id>` entry — is:

```
{
  "iterations": { "1": …, "2": …, "4": … },
  "items": [ … ],
  "count": 3,
  "terminatedBy": "condition",
  "errors": [ { "index": 3, "code": "…", "message": "…" } ]
}
```

- **`iterations`** — the `collectRef` projection (or full output) of each *successful* iteration, keyed by its **real 1-based iteration number**. `count` always equals the number of keys. A gap means the iteration did not complete successfully (if it *failed*, its number appears in `errors[].index`) — or that it succeeded but its `collectRef` didn't resolve in its output.
- **`items`** — the flattened concatenation across successful iterations: array projections are spread, scalars appended. This is usually what downstream tasks bind (`results.all_issues.items`).
- **`count`** — successful iterations.
- **`terminatedBy`** — why the loop stopped. `condition` is the only "natural" finish:

`terminatedBy`	Meaning
`condition`	The kind's own stop condition: cursor gone, empty page, item list exhausted
`bound`	A cap stopped the loop (`maxIterations`, `maxItems`, or `maxCollectedBytes`). Treat the data as **potentially partial** — though a loop whose natural end coincides exactly with the cap also reports `bound`
`error`	A `cursor`/`pages` iteration failed; the controller **fails**
`no_progress`	The same cursor token repeated (fixpoint guard) — **or** a `map` whose `itemsRef` did not resolve to an array. The second case **completes green with an empty aggregate**, so a typo'd `itemsRef` looks like success: check `count`

- **`errors`** — entries for *failed* iterations, whatever the kind. For `map` the loop continues past them; for `cursor`/`pages` the same entry appears on the failed controller's `output`.

Partial failure differs by kind. A failed `map` item becomes an `errors[]` entry and the loop *continues*; the controller still completes. A failed `cursor`/`pages` iteration **fails the controller** (`error.code:`

`LOOP_ITERATION_FAILED`): the partial aggregate is preserved on the controller's `output`, but **no `results.<id>` entry is written** — downstream `all_succeeded` dependents will not run.

The aggregate's classification is the **merge** of its iterations' classifications (provenance `source: "loop.aggregate"`) — no fresh classification pass runs over the combined value. (Exception: a loop with nothing to iterate settles through the ordinary result path and gets normal classify-on-write treatment — one classifier call over an empty aggregate.)

Runtime fields you never author

The engine annotates the controller with `loopState` (`materialized`, `settled`, `lastCursor`, `totalItems`, `foundItems`, `collectedBytes`, `terminatedBy`) and each iteration task with `loopParent` (the controller's id). Never write these — the chat-v2 patch validator rejects planner-authored `loopState` / `loopParent`, and the id suffix pattern `__i<n>` is **reserved** for engine-materialized iterations (task ids matching it are rejected). Iterations inherit the controller's `requiredCapability`, `contextMapping`, `inputSchema` / `outputSchema`, and `retry` policy — but **not** `onBehalfOf` or `dependencyCondition`.

Because the controller's `contextMapping` is re-resolved for every iteration, an `inputMappings` (or `include`) key that collides with `cursorParam` / `pageParam` / `itemParam` **overwrites the carried value at dispatch** — keep the loop's parameter field out of the mapping.

A running loop emits `loop.iteration` / `loop.terminated` events — see `WORKFLOW_EVENTS.md`.

> **Authoring-path caveats (hand-written JSON).** The loop primitive shipped on the agentic (chat-v2) patch path (HOL-2924), and that path carries ALL of its validation. `POST /workflows` shape validation knows nothing about loops: a malformed spec, an authored `loopState` / `loopParent`, or a `__i<n>`-suffixed task id are all accepted at create time and misbehave at run time instead. Three rules keep hand-authored loops on the supported path:
>
> 1. **Give the controller at least one dependency.** Loop expansion runs in the result-processor's post-result cycle (plus startup recovery and the chat-v2 dispatch path); the `POST /workflows/:id/start` root dispatch does *not* expand controllers — a root-task controller is skipped with a warning and the workflow stalls until the next result-processor restart, when the recovery sweep happens to rescue it.
> 2. **Validate the spec yourself** against the field table above (required fields per kind, `iteration.`-rooted refs, caps in range) — exactly what the patch validator would have enforced.
> 3. **Always set `maxItems` and a sane `maxConcurrency` on `map`.** The engine only *defaults* `maxIterations` and `maxCollectedBytes`; it does not clamp the other two outside the validator. A hand-authored `map` without `maxItems` is unbounded, and an oversized `maxConcurrency` materializes that many parallel iterations.
>
> The classic planner (`POST /plan`) does not emit loops at all.

---

7. Capabilities and schemas

`requiredCapability` is a namespaced type — `{owner}/{package}[.{variant}]`, e.g. `core/transform`, `core/llm.invoke`, `system/bash.exec`. It must exist in the **capability catalog**; otherwise validation fails with `CAPABILITY_NOT_IN_CATALOG`.

Each catalog entry carries an **input** and **output** JSON Schema:

```
interface HandlerCatalogEntry {
  type: string;        // "core/transform"
  input:  JsonSchema;  // shape the capability expects
  output: JsonSchema;  // shape it returns
}
```

Two things follow for you as an author:

1. **Your task's effective input must satisfy the capability's `input` schema.** Get the field names and types right for the capability you target — they come from the catalog, not from this guide.
2. **Bindings are type-checked.** For every `inputMappings` entry, the engine resolves the producer's output type (from the upstream capability's `output` schema) and confirms it's compatible with the consumer field's type. A mismatch is rejected before the task ever runs.

Schemas are **snapshotted with the workflow** at creation time, so replays use the schemas that were valid when the DAG was authored.

> **Sugar primitives are TypeScript-only.** The programmatic builder offers shorthands like `$.bash(...)`, `$.transform(...)`, `$.writeFile(...)`. These do **not** exist in the JSON contract — when hand-writing JSON, always use the longhand capability type and its full `input` object. (For reference, `$.bash` → `system/bash.exec` with `{ command, cwd?, timeoutMs?, env? }`; `$.transform` → `core/transform` with `{ expression, value, expressionType? }`; `$.writeFile` → `mcp/filesystem/write_file` with `{ path, content }`; `$.readFile` → `mcp/filesystem/read_text_file` with `{ path, head?, tail? }`; `$.echo` → `desktop/echo` with `{ message }`.)

---

8. A complete worked example

A diamond: prepare a prompt, fan out to two LLMs in parallel, fan back in to aggregate, then synthesize. (Trimmed from `docs/sample-workflow.json`; ids use the planner-safe underscore form.)

```
{
  "tasks": {
    "prepare_context": {
      "id": "prepare_context",
      "description": "Build the code-review prompt from user input",
      "requiredCapability": "core/transform",
      "dependencies": [],
      "dependencyCondition": { "type": "all_succeeded" },
      "contextMapping": { "include": ["input"], "inputMappings": {} },
      "input": {
        "expressionType": "jsonata",
        "expression": "{ 'prompt': 'Review the following code for bugs', 'codeSnippet': input
      },
      "status": "pending", "attempts": 0,
      "createdAt": "2026-01-01T00:00:00Z", "updatedAt": "2026-01-01T00:00:00Z"
    },

    "llm_anthropic": {
      "id": "llm_anthropic",
      "description": "Send the prompt to Claude",
      "requiredCapability": "core/llm.invoke",
      "dependencies": ["prepare_context"],
      "dependencyCondition": { "type": "all_succeeded" },
      "contextMapping": {
        "include": ["results.prepare_context"],
        "inputMappings": { "prompt": "results.prepare_context.prompt" }
      },
      "input": { "model": "claude-sonnet-4-20250514", "promptTemplate": "{{prompt}}", "maxTok
      "retry": { "maxRetries": 2, "backoffMs": 1000, "backoffMultiplier": 2, "maxBackoffMs":
      "status": "pending", "attempts": 0,
      "createdAt": "2026-01-01T00:00:00Z", "updatedAt": "2026-01-01T00:00:00Z"
    },

    "llm_openai": {
      "id": "llm_openai",
      "description": "Send the prompt to GPT-4",
      "requiredCapability": "core/llm.invoke",
      "dependencies": ["prepare_context"],
      "dependencyCondition": { "type": "all_succeeded" },
      "contextMapping": {
        "include": ["results.prepare_context"],
        "inputMappings": { "prompt": "results.prepare_context.prompt" }
      },
      "input": { "model": "gpt-4-turbo", "promptTemplate": "{{prompt}}", "maxTokens": 2048 },
      "status": "pending", "attempts": 0,
      "createdAt": "2026-01-01T00:00:00Z", "updatedAt": "2026-01-01T00:00:00Z"
    },

    "aggregate_responses": {
      "id": "aggregate_responses",
      "description": "Collect both responses into one array",
      "requiredCapability": "core/aggregate",
      "dependencies": ["llm_anthropic", "llm_openai"],
      "dependencyCondition": { "type": "all_completed" },
      "contextMapping": {
        "include": ["results.llm_anthropic", "results.llm_openai"],
        "inputMappings": {
          "responses.0": "results.llm_anthropic.content",
```

```
          "responses.1": "results.llm_openai.content"
        }
      },
      "input": { "strategy": "custom" },
      "status": "pending", "attempts": 0,
      "createdAt": "2026-01-01T00:00:00Z", "updatedAt": "2026-01-01T00:00:00Z"
    },

    "synthesize": {
      "id": "synthesize",
      "description": "Synthesize a unified report",
      "requiredCapability": "core/llm.invoke",
      "dependencies": ["aggregate_responses"],
      "dependencyCondition": { "type": "all_succeeded" },
      "contextMapping": {
        "include": ["results.aggregate_responses", "results.prepare_context"],
        "inputMappings": {
          "responses": "results.aggregate_responses.responses",
          "language": "results.prepare_context.language"
        }
      },
      "input": { "model": "claude-sonnet-4-20250514", "promptTemplate": "Synthesize these ana
      "status": "pending", "attempts": 0,
      "createdAt": "2026-01-01T00:00:00Z", "updatedAt": "2026-01-01T00:00:00Z"
    }
  },

  "context": {
    "input": {
      "language": "python",
      "code": "def f(x):\n    return eval(x)"
    },
    "results": {},
    "shared": {}
  },
  "artifacts": {}
}
```


Walk the graph: `prepare_context` is the root (depends on nothing, reads `input`). `llm_anthropic` and `llm_openai` each depend on it and run in parallel, both binding `results.prepare_context.prompt`. `aggregate_responses` fans in on both with `all_completed` (so it still runs if one model fails), assembling an array via nested keys `responses.0` / `responses.1`. `synthesize` depends on the aggregate and reads back two upstream outputs.

---

9. Validation and common errors

Your definition is validated against the TypeBox `WorkflowDefinitionSchema` on `POST /workflows`, and binding/type checks run when the plan is finalized. What's checked:

- **Shape** — `tasks`, `context` (`input` / `results` / `shared`), and `artifacts` present; each task has its required fields; `id` matches its map key.
- **Acyclicity** — no cycles in the dependency graph.

- **Reference integrity** — every id in `dependencies` and every `results.<id>` in a binding names a real task.
- **Capability presence** — every `requiredCapability` exists in the catalog.
- **Binding types** — each `inputMappings` producer type is compatible with the consumer field type.

Error codes you'll see, and what they mean:

Code	Cause	Fix
`CAPABILITY_NOT_IN_CATALOG`	`requiredCapability` isn't a known type	Use a catalog type; check spelling/namespace
`INVALID_CONTEXT_PATH_ROOT`	binding path doesn't start with `results` / `input` / `shared`	Re-root the path
`CONTEXT_PATH_NOT_FOUND`	dot-path doesn't resolve against the producer's output	Fix the field name / path
`CONSUMER_INPUT_KEY_UNKNOWN`	the capability's input schema has no such field	Match the catalog input field names
`TYPE_MISMATCH`	producer output type ≠ consumer input type	Insert a `core/transform`, or fix the mapping

> Nested consumer keys (`value.user`) skip the consumer-key existence check on the *key* but still type-check the producer side — so verify the capability really accepts a nested object of that shape.

---

10. Parameterizing with templates

There is **no placeholder/variable syntax inside the JSON** (no `${var}`). DAGs are parameterized through the **context input** plus the template layer:

- **Author for reuse** by reading every variable input from `input.<field>` (bind it through `contextMapping` , never hard-code it in a task's static `input`).
- **Save a template** (`POST /workflows/:id/templatize`): the engine strips runtime state (statuses, attempts, results) and generates an `input_schema` by scanning your `context.input.*` references.
- **Launch from a template** (`POST /workflows/from-template`) with an `inputs` object; it's validated against the template's `input_schema` (JSON Schema draft 2020-12), becomes `context.input` , and the DAG runs as normal.

So the path to a parameterized DAG is: bind everything variable from `input.*` → templatize → launch with `inputs` .

---

11. Authoring checklist

Before you submit a workflow definition, confirm:

- [ ] **`tasks`, `context` (`input` / `results` / `shared`), `artifacts` all present**; `results` and `shared` start as `{}`.
- [ ] **Every `tasks` key equals its task's `id`.**
- [ ] **Every task sets `requiredCapability`** to a real catalog type.
- [ ] **`dependencies` lists only real task ids**; roots use `[]`.
- [ ] **No cycles.**
- [ ] **`dependencyCondition` matches intent** — `all_completed` for fan-ins that tolerate failure; `all_succeeded` otherwise.
- [ ] **Every binding path is rooted** at `results` / `input` / `shared` and resolves to a real field.
- [ ] **Static `input` and bindings don't collide** unintentionally (bindings win).
- [ ] **Variable values come from `input.*`**, not hard-coded, if you'll templatize.
- [ ] **Initial state set**: `status: "pending"`, `attempts: 0`, timestamps present; no `output` / `error` /lease fields.
- [ ] **Loop controllers** (if any): required fields for the kind present, bounds within the platform caps, refs correctly rooted (`iteration.` / context roots), **at least one dependency**, and no authored `loopState` / `loopParent` or `__i<n>`-suffixed ids. See §6.

---

12. Gotchas

- **Map key vs `id`.** They must match exactly. This is the most common hand-authoring mistake.
- **`context` is not optional.** Even an all-root workflow needs `"context": { "input": {}, "results": {}, "shared": {} }` — the engine writes results into it.
- **Don't pre-fill `results`.** Start it empty; the engine owns it.
- **Hyphens vs underscores in ids.** Legal for the core engine, but the agentic planner rejects hyphens — prefer underscores for forward-compatibility.
- **Sugar isn't JSON.** `$.bash(...)` etc. are TypeScript builder helpers; hand JSON uses the longhand capability + full input.
- **Capability input shapes are catalog-defined.** The field names in this guide's examples are illustrative — always check the catalog entry for the exact `input` schema of the capability you target.
- **Type mismatches fail before execution.** If two steps don't line up, insert a `core/transform` to reshape rather than forcing the binding.
- **`__i<n>` is a reserved id suffix.** The engine materializes loop iterations as `<controllerId>__i1`, `__i2`, …; don't name your own tasks that way.
- **A loop controller must not be a root task.** Root dispatch on `/start` skips controllers; expansion happens in the result-processor cycle, so a root controller stalls the workflow until the next processor restart. Give every controller at least one dependency. See §6.

Where to go next

- **The capability types and their schemas** → `CAPABILITY_CATALOG.md`

- **How a task is routed and executed** → `NEURON_ARCHITECTURE.md`
- **A full untrimmed example** → `sample-workflow.json`
- **Lifecycle events to observe a run** → `WORKFLOW_EVENTS.md`